



Estruturas de Dados na Linguagem C

Introdução a Structs e Registros para Representação de Tipos Abstratos de Dados

Conceitos

Registros (Structs)

Um registro é uma variável especial que contém diversas variáveis.

Na linguagem C, os registros são especificados pela palavra-chave '**struct**'.

As variáveis internas contidas pela struct são denominadas membros ou campos.

As structs permitem agrupar dados distintos, sendo essenciais para representar e organizar tipos abstratos de dados complexos em um programa.

REPRESENTAÇÃO EM MEMÓRIA (STRUCT)

char[50]	nome
float	nota_prova1
float	nota_prova2

Declaração de Registros

A declaração de um registro em C exige a palavra-chave **struct** seguida de um identificador que nomeia a estrutura.

O corpo da estrutura define o molde de dados, sendo delimitado por chaves e listando os tipos e identificadores dos membros internos.

FORMATO GENÉRICO

```
struct <identificador> {  
    <tipo> <membro_1>;  
    <tipo> <membro_2>;  
};
```

EXEMPLO DE CÓDIGO C

```
struct ficha_de_aluno {  
    char nome[50];  
    float nota_prova1;  
};
```

ATENÇÃO / SINTAXE

O encerramento do corpo de uma estrutura com **ponto e vírgula (;)** após o fechamento da chave é estritamente **obrigatório**.

Esquecer o ";" após a chave de fechamento ("};") é um dos erros de compilação mais frequentes.

Exemplo Prático: `struct ficha_de_aluno`

Agrupamento de Dados Mistos

Considere o exemplo de uma estrutura projetada especificamente para armazenar dados escolares de forma unificada.

A **`struct ficha_de_aluno`** define um molde que agrupa diferentes tipos de dados essenciais:

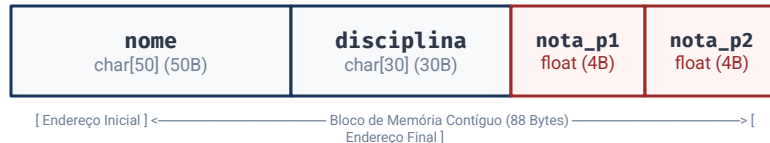
- **`nome`** e **`disciplina`**: Matrizes de caracteres (*strings*) que armazenam as informações de identificação textual.
- **`nota_prova1`** e **`nota_prova2`**: Variáveis numéricas decimais (*float*) para os registros de notas.

Este molde especifica exatamente quais informações compõem o registro acadêmico, organizando múltiplos atributos na mesma unidade estrutural lógica.

Definição da Estrutura

```
struct ficha_de_aluno {  
    char nome[50];  
    char disciplina[30];  
    float nota_prova1;  
    float nota_prova2;  
};
```

Representação Teórica de Alocação em Memória



Declaração de Variáveis

Depois de criar a **struct**, precisamos criar a variável que vai utilizá-la.

A declaração padrão utiliza a sintaxe:

```
struct <identificador> <variavel>;
```

Por exemplo, para instanciar a estrutura previamente definida, utilizamos o comando:

```
struct ficha_de_aluno aluno;
```

Este comando cria a variável **aluno**, que será do tipo **ficha_de_aluno**, alocando espaço na memória para todos os membros definidos no molde original.

EXEMPLO DE CÓDIGO C

```
// 1. Definição do molde estrutural
struct ficha_de_aluno {
    char nome[50];
    float nota_prova1;
};

// 2. Criação física da variável em memória
struct ficha_de_aluno aluno;
```

Ordem Lógica de Declaração

Definição global da estrutura e instanciação local de variáveis em C

No desenvolvimento de um programa em C, a declaração da estrutura e de suas variáveis segue uma ordem lógica. Primeiro, criamos a **struct** de forma global. Em seguida, dentro da função `int main(void)`, criamos a variável que será do tipo correspondente.

EXEMPLO DE ESTRUTURAÇÃO DE CÓDIGO FONTE (C)

```
// 1. Primeiro: Criamos a struct (Escopo Global)
struct ficha_de_aluno {
    char nome[50];
    char disciplina[30];
    float nota_prova1;
    float nota_prova2;
};

// 2. Depois: Declaramos a variável do tipo struct dentro da função main
int main(void) {
    struct ficha_de_aluno aluno;

    return 0;
}
```

Leitura de Membros de Structs

O Operador de Ponto (.)

A leitura de dados para os membros da struct exige o uso do **operador de ponto** ('.'). Ele é fundamental para vincular a variável estruturada ao seu respectivo campo interno na memória.

Fluxo de Captura de Dados

Leitura de Strings: Para ler strings da entrada padrão (stdin), utiliza-se o comando `fgets(aluno.nome, 40, stdin)`.

Limpeza do Buffer: É essencial preceder a leitura de strings com o comando `fflush(stdin)` para limpar o buffer do teclado.

Dados Numéricos: Emprega-se a função padrão `scanf("%f", &aluno.nota_prova1)`, referenciando diretamente o endereço do membro.

EXEMPLO DE LEITURA EM C

```
// Limpeza preventiva de buffer
fflush(stdin);

// Leitura segura de string para campo
fgets(aluno.nome, 40, stdin);

// Captura de valor numérico
scanf("%f", &aluno.nota_prova1);
```

ALERTA DE SEGURANÇA E SINTAXE

O comando `fflush(stdin)` evita que caracteres órfãos (como `\n`) remanescentes no buffer façam com que o `fgets` seja ignorado pelo sistema operacional.

O operador de ponto conecta fisicamente o registro de memória do aluno ao campo requisitado.

Exibição de Registros

Uso da função printf e acesso a membros com o operador de ponto

Acesso e Saída de Dados

Para exibir os valores armazenados no registro, utilizamos a função `printf`, acessando cada campo individualmente através do operador de ponto.

Regras de Formatação

◆ **Exibição de Strings:** Exemplo de exibição de strings:
`printf("Nome ... : %s", aluno.nome);`.

◆ **Membros Decimais:** Para formatar e exibir os membros numéricos decimais, utilizamos os especificadores adequados: `printf("Nota ... : %.2f\n", aluno.nota_prova1);`.

◆ **Variáveis Independentes:** Cada membro interno é tratado e impresso como uma variável independente.

EXEMPLO COMPILÁVEL EM C

```
#include <stdio.h>

struct FichaAluno {
    char nome[50];
    float nota_prova1;
};

int main() {
    struct FichaAluno aluno;

    /* Exibição usando operador de ponto */
    printf("Nome ... : %s", aluno.nome);

    /* Formatação de membro decimal */
    printf("Nota da Prova 1 ... : %.2f\n",
        aluno.nota_prova1);

    return 0;
}
```

Vetores de Structs

Agrupamento de Registros Complexos

Uma struct pode ser associada a vetores, permitindo a criação de um **vetor de structs**.

Esta abordagem é idêntica à criação de um vetor de tipos primitivos (como inteiros), mas cada posição do array armazena um registro completo na memória com múltiplos campos.

Aplicações Práticas

É uma técnica extremamente útil para cadastrar múltiplos registros do mesmo tipo em lote.

Exemplo: Armazenar os dados de uma classe inteira com até 50 alunos em uma única estrutura de dados estruturada.

Representação em Memória (Vetor turma[50])

turma[0]

char[50]	nome = "Ana Silva"
float	nota = 9.5

turma[1]

char[50]	nome = "Bruno Costa"
float	nota = 8.0

...

turma[49]

char[50]	nome = "Zeca Souza"
float	nota = 7.5

Vetores de Estruturas

Para declarar um vetor de estruturas, especificamos a quantidade de elementos no momento da criação da variável.

Utilizando o molde de alunos, a sintaxe em C é:

```
struct ficha_de_aluno turma[50];
```

Este comando cria uma variável chamada **turma**, que é um vetor contendo 50 registros do tipo **ficha_de_aluno**.

O computador aloca a memória necessária para armazenar todos esses elementos sequencialmente.

Exemplo de Código C

```
struct ficha_de_aluno {  
    char nome[50];  
    float nota_prova1;  
};  
  
// Alocação sequencial (em lote)  
struct ficha_de_aluno turma[50];
```

Visualização Teórica de Alocação em Lote



Armazenamento Sequencial Contíguo

Acesso e Iteração em Vetores de Structs

PROG. ESTRUTURADA

O acesso aos campos de um vetor de structs combina colchetes de indexação e o operador de ponto. Para fazer referência ao campo **nome** do primeiro aluno, usa-se a notação **turma[0].nome**.

A iteração por todos os registros é feita com laços de repetição tradicionais, onde o índice do vetor pode ser representado por uma variável de controle.

EXEMPLO DE IMPLEMENTAÇÃO EM C

```
#define NumAlunos 50
struct Aluno turma[NumAlunos];
int i;

// Iteração por todos os registros do vetor com exibição dos campos
for (i = 0; i < NumAlunos; i++) {
    printf("Nome: %s\n", turma[i].nome);
}
```

Estruturas Aninhadas em C

Agrupamento lógico de dados compostos (nested structs)

Modularidade e Organização

A linguagem C permite declarar uma estrutura como membro de outra. Esse conceito de **estruturas aninhadas** é amplamente utilizado para agrupar logicamente dados complexos correlacionados.

- ◆ **Conceito Chave:** Uma struct independente (ex: Endereco) é declarada primeiro e depois inserida diretamente como membro de uma struct principal (ex: Aluno).
- ◆ **Reusabilidade:** A struct secundária torna-se um bloco modular reutilizável, podendo ser inserida em múltiplos outros cadastros do sistema de forma limpa.
- ◆ **Acesso Encadeado:** O acesso aos campos internos é feito de forma encadeada utilizando operadores ponto consecutivos (ex: aluno.endereco.cep).

DIAGRAMA: ANINHAMENTO DE MEMÓRIA

struct Aluno

```
char nome[50];
```

```
int matricula;
```

struct Endereco endereco;

rua
char[100]

bairro
char[50]

cep
char[10]

Figura: A subestrutura **Endereco** é instanciada internamente como membro de **Aluno**.

Estruturas Aninhadas & Encadeamento

Abstração & Hierarquia

Em C, a modularidade de dados é obtida aninhando registros. Uma estrutura de dados de nível superior pode conter campos que são, por si só, outras estruturas completas.

No modelo ao lado, a **struct aluno** incorpora a **struct ender**, unificando os dados básicos de identificação e localização do estudante em uma única representação lógica.

REFEITORIA DE DADOS E ACESSO

Para acessar membros internos de structs aninhadas, utiliza-se o encadeamento pelo **operador duplo de ponto**:

```
cadastro[i].endereco.rua
```

A resolução é feita de fora para dentro: identifica-se o índice do vetor, acessa-se o registro intermediário e, enfim, o dado interno.

EXEMPLO PRÁTICO EM C

```
struct ender {  
    char rua[50];  
    int numero;  
};  
  
struct aluno {  
    int matricula;  
    struct ender endereco;  
};  
  
// Declaração do vetor de structs  
struct aluno cadastro[50];  
  
// Acesso e atribuição via ponto duplo  
cadastro[0].endereco.numero = 101;  
strcpy(cadastro[0].endereco.rua, "Rua das Flores");
```

Registros (Structs)

Agrupamento de variáveis heterogêneas em unidades lógicas

Pontos-Chave do Conteúdo

- ◆ **Sintaxe de Declaração:** Definição de novos tipos de dados estruturados com a palavra-chave `struct`.
- ◆ **Acesso a Membros:** Utilização direta e intuitiva do operador de ponto para manipulação de campos internos.
- ◆ **Vetores de Registros:** Declaração e iteração de conjuntos homogêneos de dados para múltiplos cadastros.
- ◆ **Aninhamento de Structs:** Organização lógica modular de dados e estruturas hierárquicas complexas.

Boas Práticas de Programação

O controle estrito dos buffers de entrada é indispensável para evitar falhas graves de execução ao manipular cadeias de caracteres.

Atenção ao Buffer: Lembre-se sempre de realizar a limpeza do buffer de entrada com o comando `fflush(stdin)` logo antes de efetuar leituras de strings utilizando `fgets`.

Esta prática previne que resíduos de caracteres de comandos anteriores comprometam a integridade dos dados capturados.

Síntese

Declaração do tipo

```
struct Aluno
{
    char nome[50];
    int matricula;
    char nascimento[12];
    char sexo;
    char telefone[20];
};
```

Instanciação

Permite usar o tipo **Aluno** para criar variáveis.

```
struct Aluno a1 = {  
    "Ana Maria",  
    20200101,  
    "15/04/2002",  
    'F',  
    "(11) 98765-4321"  
};
```

Acesso aos campos

Operador .

```
a1.matricula = 20250101;
```

Declaração com typedef

```
typedef struct  
{  
    float peso;  
    float altura;  
} Pessoa;
```